

anyone other than the user himself. To ensure this they encrypt all sensitive user data.

However all data will eventually need to be decrypted in order to be used by a user of the site. Improper access control measures can lead to the leakage of such data if the mechanism used to store and retrieve them is not proper.

Imagine a hypothetical system where credit card numbers are stored in a table for use by a web application. To ensure that the credit card numbers will be secure in case of a database leak, the values are stored in encrypted form in the database. However, the application is set to decrypt these entries automatically when requested by the server. In this case, an attacker could use a SQL injection vulnerability to expose the credit card numbers.

As you will learn in the section about passwords and hashes, an unsalted hash is vulnerable to decryption. If the password for the users is being stored as an unsalted hash, accidental exposure of these passwords via a vulnerability will be much more harmful.

Another example – which parallels writing your computers administrator password on a sticky note on you monitor – is to actually leave your password exposed in the same volume as an encrypted backup! To save from remembering yet another password, the person conducting the backup might store the password in; say the volume label of the DVD with the backup on it. It is clear that simply using encryption is not enough, the decryption procedure, and key need to be kept secure as well.

3.2.9 Failure to restrict URL access

“Many web applications check URL access rights before rendering protected links and buttons. However, applications need to perform similar access control checks each time these pages are accessed, or attackers will be able to forge URLs to access these hidden pages anyway.”

Most websites will be careful about restricting the entry points of their administrative or restricted interfaces. So a website will likely restrict access to URLs for their administrative interfaces with login pages, but what about what lies under them?

For example, a web site <http://www.somesite.com> has an administrative section of the site under the “/admin” directory which allows them to add, remove and update content on the website. The administration section is guarded with a password. To make things easier for those working on the site, the owners of **somesite.com** decided to install a file management application to the backend. This application would give an administrator full control of the websites files. Unfortunately, this application is not well integrated with the authentication mechanism of the site, and attackers find that by accessing the URL for the file browser directly at “<http://www.somesite.com/admin/filebrowser>”, without going through the administrative